CSS326 Project Proposal

Project name:

Cross-referencing media universe

Group Members

1. Mr. Wayupuk Sommuang 6722782388 Project Manager, backend design
2. Ms. Chanidapa Thongbor 6722782735 User Flow, Frontend design
3. Ms. Sirikorn Nukpook 6722782981 Database design

Project Description

This website is designed for movie, series, and comic fans who want to discover whether their favorite titles have additional adaptations beyond the one they're currently watching or reading.

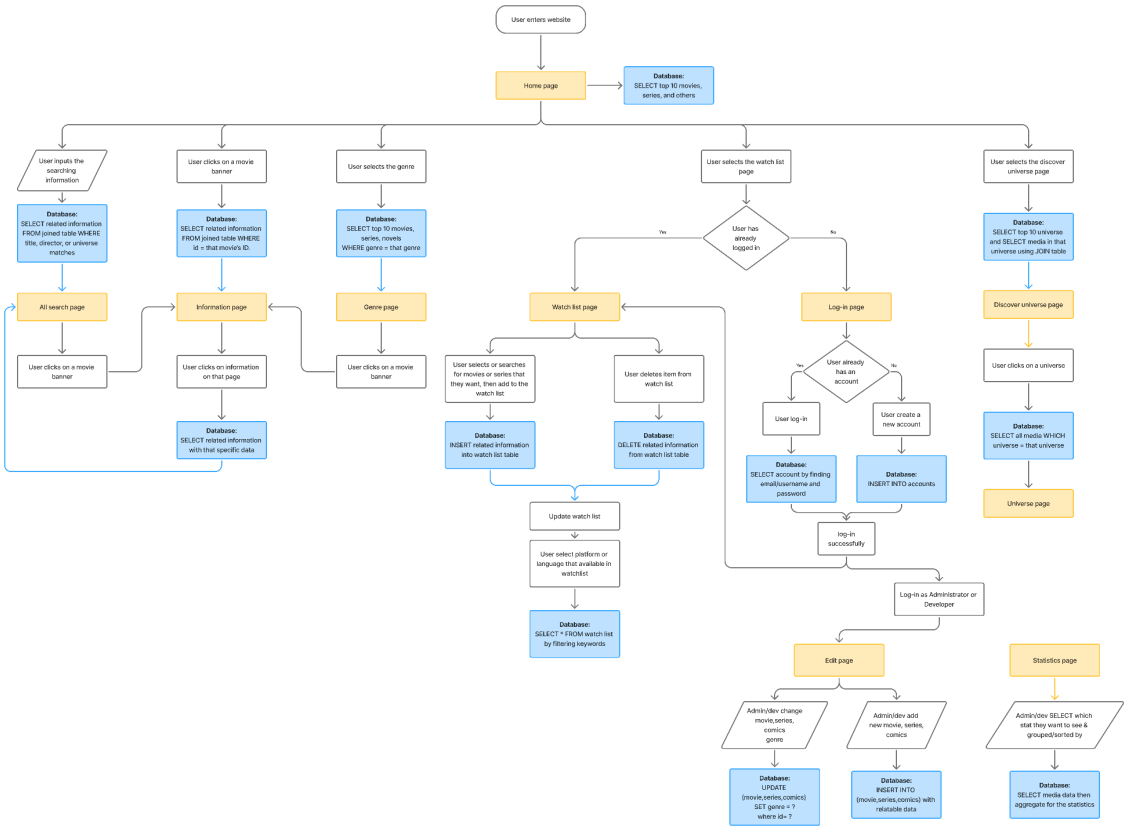
The target audience includes people who are passionate about movies, series, and comics and want to explore more about their current favorites to further satisfy their interest.

This website reduces the time users spend manually searching the internet, and avoids the incomplete coverage that can come with relying on AI-generated answers.

Users and Roles (access control)

Role	Who	What they can do
User	Normal user	Can search and select the movie
Administrator	The person who manage contents and inspect user data	Add or remove data in websites and access some user’s data (what movie gets the most search etc.)
Developer	Our team	Access the full database system, change router endpoint, change website format

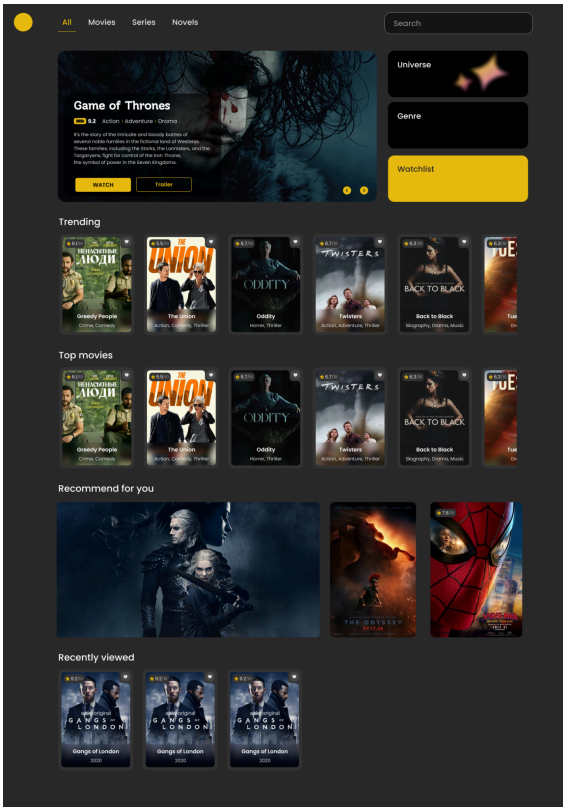
User Flows



1. **User search** → User give information → backend preprocess text for DB → DB querry
2. **User click Movie banner** – User click the banner → backend redirect user to the target page
3. **User select genre** → select genre → backend send selected genre to DB to re-querry → DB receive the genre and run SELECT argument
4. **Add to watchlist** → user add interested ones to watchlist → check if user has Logged in. if Logged in pass the value to DB based on the customer. → DB INSERT value to specific table with where argument
5. **User create account** → create account → backend will receive argument and pass to DB → DN will run INSERT in customer table
6. **User delete movie from watchlist** → backend receive the argument and pass to DB → DB run DELETE command base on movie id from backend
7. **Administrator add new movie, series or comic** → backend receive admin request → DB INSERT value from backend
8. **Administrator change movie, series or comic genre** → backend receive new genre → DB UPDATE target table via id
9. **User select universe** → user discover universe page → backend receive request → DB query data with SELECT for top 10
10. **Administrator / developer inspect stats** → Admin/dev select statistics page → backend receive request → DB SELECT data then send back to backend for aggregate

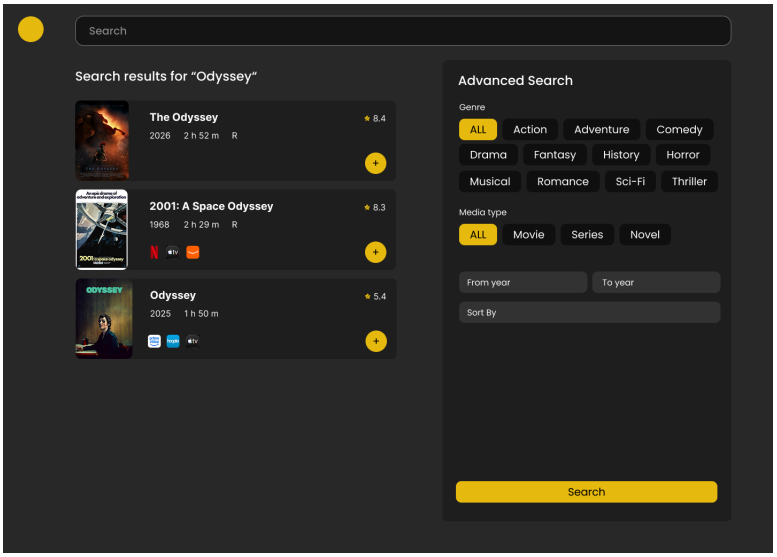
User Interface

Page 1: Home page – used by everyone



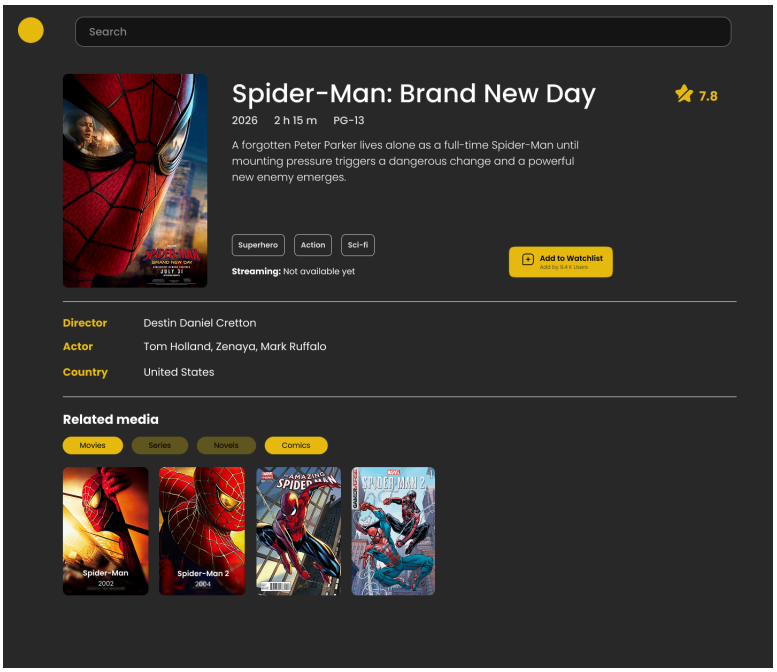
This page reads data from Movies, Series, Novels joined table (Media).

Page 2: All Search page – used by everyone



This page reads data from Movies, Series, Novels joined table (Media). Using keywords from each attribute to filter the results.

Page 3: Information page – used by everyone

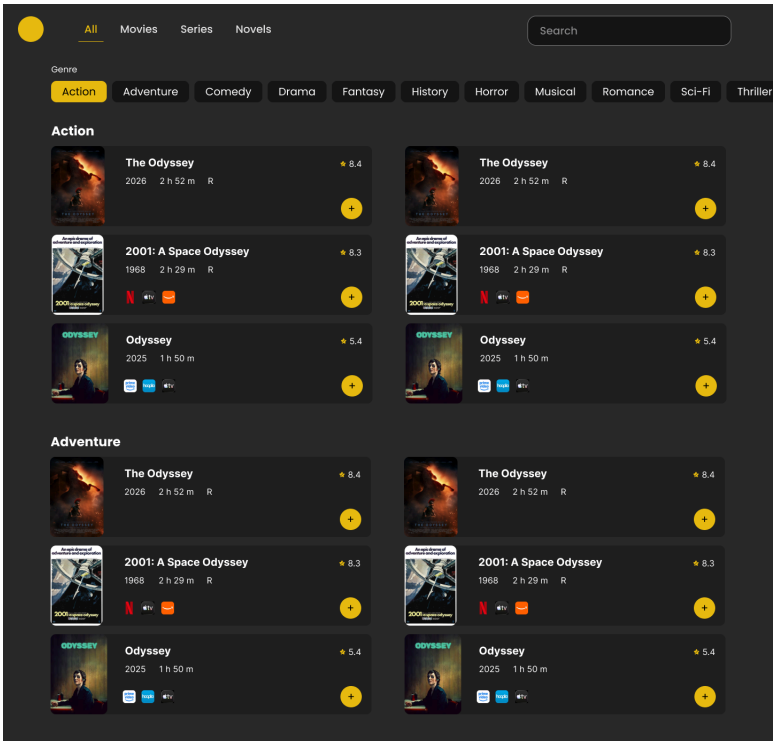


This page reads data where id = specific key from Movies, Series, Novels joined table.

Link data to the universe with the match id

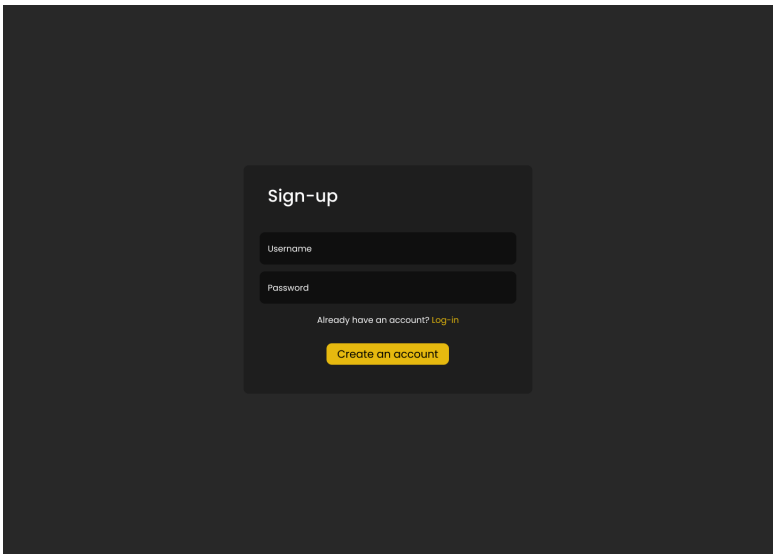
The administrators and developers can update data on this page by updating the element in the related table.

Page 4: Genre selection page – used by everyone



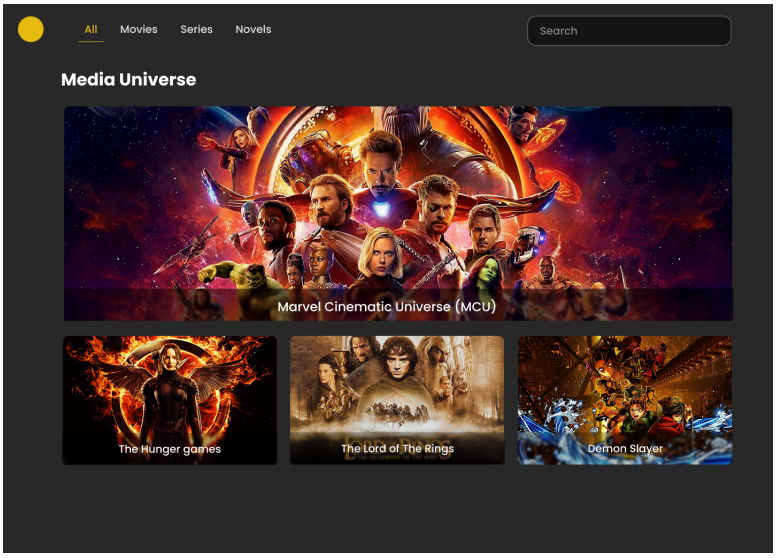
This page reads data from Movies, Series, Novels joined table (Media). Grouping the results by genre attribute.

Page 5: Log-in page – used by everyone



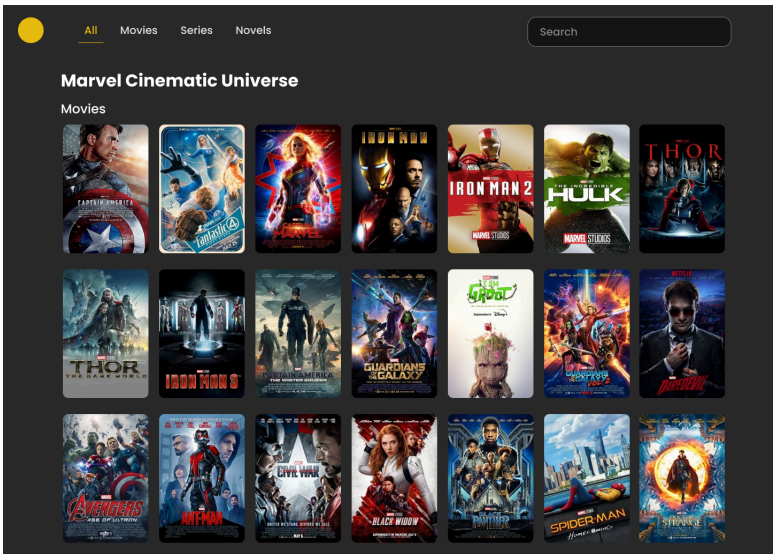
If the user does not already have an account, this page creates data into the Accounts table. If the user already has an account and wants to log-in, this page reads data from the Accounts table.

Page 6: Discover Universe page – used by everyone



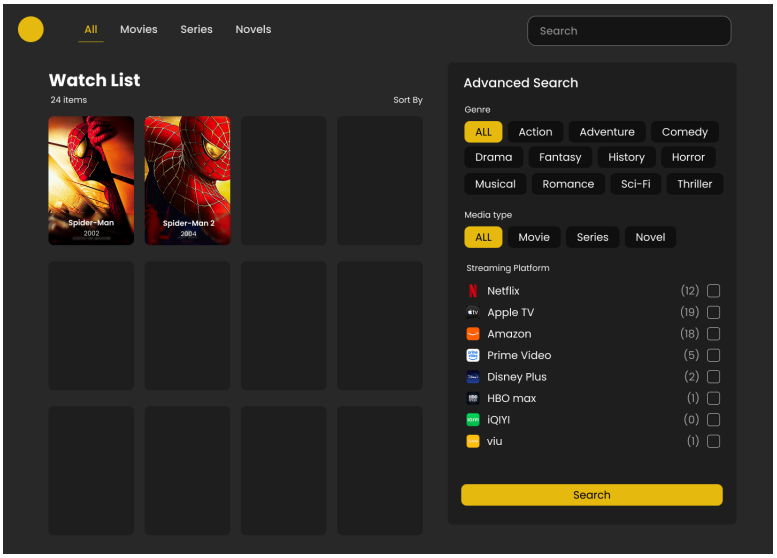
This page reads data from the Universes table and joins with the other tables for more information.

Page 7: Universe page – used by everyone



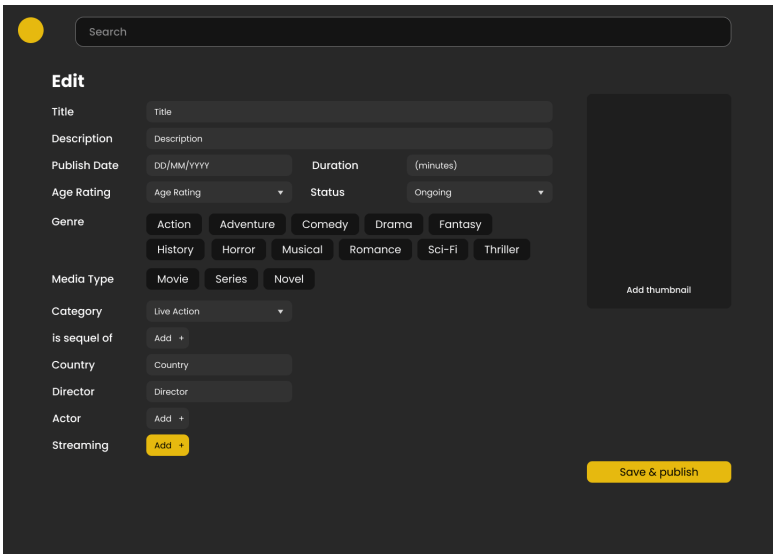
This page reads data from the Universe and the other tables by using id_universe to find all the related media in that universe.

Page 8: Watch list page – used by everyone



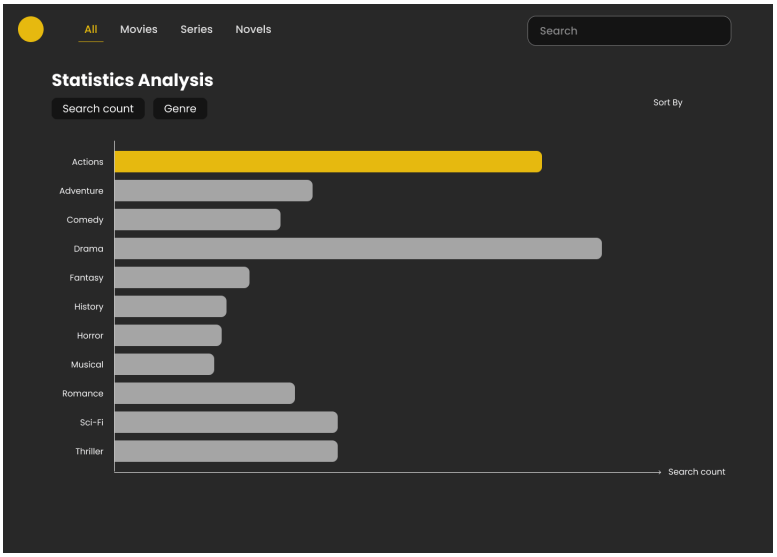
This page inserts data into the Watchlists table once the user adds an item into the watchlist, reads data when user opens this page, and deletes data when user removes item from the watchlist.

Page 9: Edit page – used by administrator



This page allows the administrator and developer to access only. They can create, update, or delete all the data in movies, series, or novels and add thumbnail pictures on this page.

Page 10: Statistics page – used by administrator and developer



This page allows the administrator and developer to access only. This page shows the website statistics about users such as search count, view count, or individual profile of each media. Selected, sorted, and grouped by custom feature for further analysis.

Database Design

Entities list

User - disjoint subclass **- Developer, Admin, Normal User**

- **user** – accountID (PK), username, password, gmail
- **developer** – accountID (PK,FK), contract
- **admin** – accountID (PK,FK), contract
- **normal user** – accountID (PK,FK)

media - disjoint subclass - **movies, series**

- **media** – id (PK), title, publish_date, age_rating, status, synopsis, category, country, rating
- **movies** – id (PK,FK), duration
- **series** – id (PK,FK), episode_number

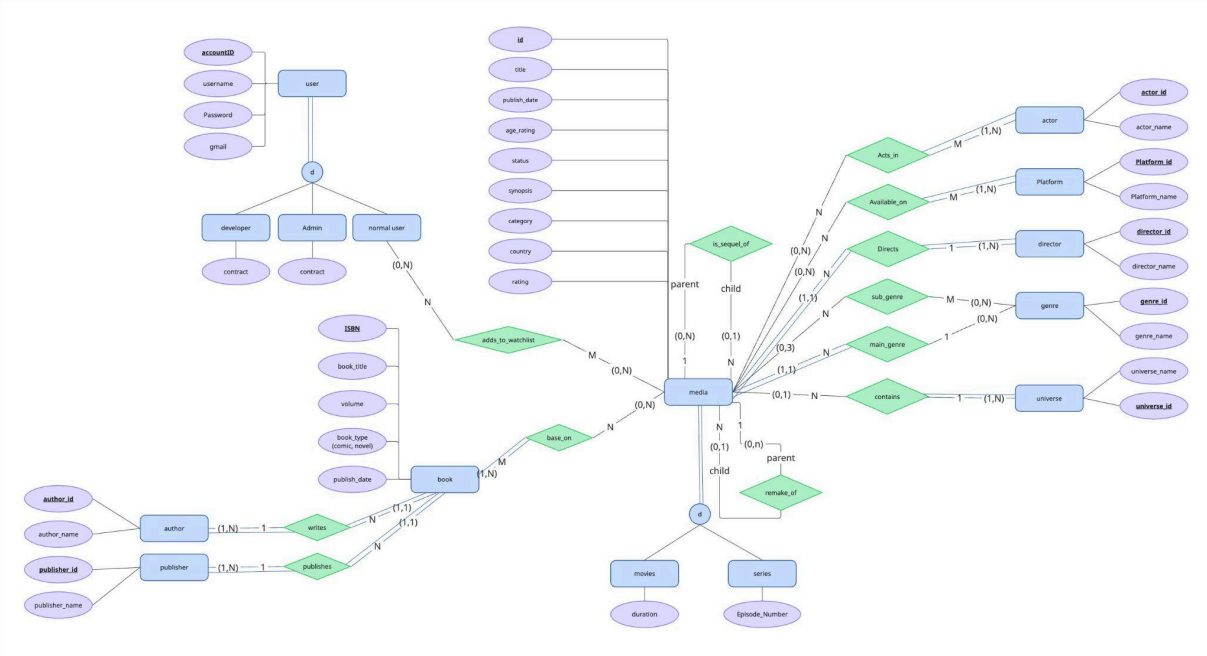
- **universe** – universe_id (PK), universe_name
- **book** – ISBN (PK), book_title, volume, book_type, publish_date,
- **author** – author_id (PK), author_name
- **publisher** – publisher_id (PK), publisher_name
- **actor** – actor_id (PK), actor_name
- **platform** – platform_id (PK), platform_name
- **director** – director_id (PK), director_name
- **genre** – genre_id (PK), genre_name

Relationship sets

1. Developer, Admin, and Normal User are disjoint subtypes of User.
2. Movies, Series are a disjoint subtype of Media
3. Normal user edit in watchlist Media (M:N)
4. Universe contains Media (1:N)
5. Media is based on Book (M:N)
6. Author writes Book (1:N)
7. Publisher publish Book (1:N)
8. Media (child) is sequel of Media (parent) (1:N)
9. Media (child) is remake of Media (parent) (1:N)
10. Actor acts in Media (M:N)
11. Media is available on Platform (M:N)
12. Director directs Media (1:N)
13. Media has sub_genre Genre (M:N)
14. Genre is the main genre of Media (1:N)

ER Diagram

PDF version: DB_project_ER.pdf



Physical Design for MySQL

Title	*Column*	*Data type*	*Nullable*	*Key*	*References*	*Notes*
User	accountID	accountID INT auto increment	No	PK		username and gmail have to be unique
	username	username varchar(60)				
	password	password varchar(60)				
	gmail	gmail varchar(100)				
Developer	accountID	accountID INT	No	PK,FK	user.accountID	username and gmail have to be unique
	contract	contract varchar (250)				
Admin	accountID	accountID INT	No	PK, FK	user.accountID	username and gmail have to be unique
	contract	contract varchar (250)				

Title	*Column*	*Data type*	*Nullable*	*Key*	*References*	*Notes*
Normal user	accountID	accountID INT	No	PK, FK	user.accountID	
Universe	universe_id	Universe_id INT auto increment	No	PK		
	universe_name	universe_name varchar (100)				
Media	id	id INT auto increment	No	PK		
	title	title varchar (100)				
	publish_date	publish_date DATE				
	age_rating	age_rating varchar(10)				
	status	status varchar(20)				
	synopsis	synopsis varchar (255)				
	category	category varchar(50)				
	country	country varchar(50)				
	rating	rating decimal (2,1)				
	universe_id	universe_id INT		FK	universe.universe_id	
	director_id	director_id INT		FK	director.id	
	genre_id	genre_id INT		FK	genre.genre_id	
	sequel_parent_id	sequel_parent_id INT		FK	media.id	
	remake_parent_id	remake_parent_id INT		FK	media.id	
Movies	id	id INT	No	PK, FK	media.id	Duration is kept in minute
	duration	duration decimal(5,2)				
Series	id	id INT	No	PK, FK	media.id	

Title	*Column*	*Data type*	*Nullable*	*Key*	*References*	*Notes*
	episode_number	episode_number int				
Books	ISBN	ISBN varchar (20)	No	PK		
	book_title	book_title varchar(100)				
	volume	volume int				
	book_type	book_type varchar(50)				
	publish_date	publish_date DATE				
	publisher_id	publisher_id INT		FK	publisher.publisher_id	
	author_id	author_id INT		FK	author.author_id	
author	author_id	author_id INT auto increment	No	PK		
	author_name	author_name varchar (100)				
publisher	publisher_id	publisher_id INT auto increment	No	PK		
	publisher_name	publisher_name varchar (100)				
actor	actor_id	actor_id INT auto increment	No	PK		
	actor_name	actor_name varchar (100)				
platform	platform_id	platform_id INT auto increment	No	PK		
	platform_name	platform_name varchar (45)				
director	director_id	director_id INT auto increment	No	PK		
	director_name	director_name varchar (100)				
genre	genre_id	genre_id INT auto increment	No	PK		

Title	*Column*	*Data type*	*Nullable*	*Key*	*References*	*Notes*
	genre_name	genre_name varchar (45)				
subgenre	id	id INT	No	PK, FK	media.id	
	genre_id	genre_id INT		PK, FK	genre.genre_id	
watchlist	accountID	accountID INT	No	PK, FK	normal_user.accountID	
	id	id INT		PK, FK	media.id	
media_actor	id	id INT	No	PK, FK	media.id	From Acts_in relationship
	actor_id	actor_id INT		PK, FK	actor.actor_id	
media_platform	id	id INT	No	PK, FK	media.id	From Available_on relationship
	platform_id	platform_id INT		PK, FK	platform.platform_id	
media_book	id	Id INT	No	PK, FK	media.id	
	ISBN	ISBN varchar (20)		PK, FK	book.isbn	

CRUD Plan

Operation	Page	Table(s)	What the person does
Create (INSERT)	Log-in, Information page, Watchlist page	User, media, watchlist	Add new user to user table, Add new media, Add item into watchlist
Read (SELECT)	Search page, Home page, Information page	Media (Joined table from movie, comics, series)	Search from a given information
Update	Information page	Media (Joined table from movie, comics, series)	Only Administrator and Developer Update comics, series, movies' genre
Delete	Watchlist page	watchlist	Delete comics, series, movies from watchlist

Technology, Stored Procedure and Trigger

Layer	Choice	Notes
Front end	HTML and CSS	HTML and CSS is a basic start for Frontend

Server	Node.js with Express	-
Database	MySQL 8	mysql2/promise with ? placeholders on every query
Stored procedure	Creating a universe for adaptations movie, series, comics for fast query	With universe queries, the website can search data faster and easier from a programming perspective.
Trigger	Automatically update if new object have more adaptation	It need to make sure that it update the right object
Security	User password will be collected in encrypted form to enforce the right user protection	User.password are hashed. Only developer and administrator with priority can access this data to prevent data leak

Timeline

Weeks	**Month**	**Date**	**Plan**
Week 7	Sep	18	**Project proposal: report submitted and presented (Lab 7)**
Week 8		25	Midterm week (no class)
Week 9	Oct	2	Prepare, create and recheck data
Week 10		9	Create UI to link to user flow while backend works with data design
Week 11		16	Link backend to database and frontend
Week 12		23*	Test and validate results, then implement corrections
Week 13		30	Test and validate results, then implement corrections
Week 14	Nov	6	**Project Demonstration (Lab 13)**
Week 15		13	No Lab (Final Exam)

** 23 October is tentative (Chulalongkorn Day).*